EVOCOWORK: A BENCHMARK FOR EXPERIENCE-DRIVEN EVOLUTION OF AGENT HARNESSES ON REAL-WORLD OFFICE TASKS

Anonymous authors
Paper under double-blind review

ABSTRACT

Most agent benchmarks score a frozen model–harness pair, even though prompts, tools, memory, middleware, and control flow strongly shape the resulting behavior. We introduce EVOCOWORK, an artifact-centric benchmark for measuring whether an agent harness can improve from experience while its underlying model remains fixed. The benchmark specifies 240 tasks across presentation, document, spreadsheet, and cross-office workflows, partitioned into 80 feedback, 40 selection, and 120 sealed final tasks by leakage group. Each task is packaged as a replayable execution unit with source-traceable inputs, an explicit deliverable contract, atomic rubrics, and artifact-aware verification. The protocol fixes the solver and updater models, tool interface, resource budget, and trusted control plane; only policy-declared harness artifacts may change. We also introduce Group-Relative Harness Search (GRHS), a reference method that samples groups of discrete harness patches, scores their held-out utility relative to sibling candidates, and promotes or rolls back revisions without updating model parameters. When the updater's own instructions, skills, or workflow are included in the mutable state, accepted revisions can alter how later revisions are generated, yielding an operational form of harness-level recursive self-improvement. **[MOCK: Across P fixed-model configurations, GRHS improves sealed-final performance by X.X percentage points over the initial harness and by Y.Y points over the strongest matched-budget baseline.]** EVOCOWORK therefore evaluates not only whether an office agent works, but whether its harness learns to work better without access to the final tasks.

1 INTRODUCTION

The capability of a language-model agent is not determined by model weights alone. A model is embedded in an *agent harness*: the instructions, skills, memory, middleware, hooks, tools, and control flow that translate token predictions into actions. Recent systems therefore treat the harness itself as an optimization target, using execution traces to revise prompts, workflows, or executable runtime components (Robeyns et al., 2025; Lin et al., 2026; Chen et al., 2026). This shift raises a question that static leaderboards cannot answer: given the same model and the same budget, which evolution procedure produces a harness that generalizes better after learning from experience?

The question is especially demanding in office work. A useful office agent must inspect heterogeneous files, preserve unrequested content, reconcile evidence across sources, and return editable documents whose formulas, layout, provenance, and visual quality all matter. Its failures are correspondingly diagnostic: a broken spreadsheet formula, an unsupported statement, or a slide that violates a requirements document points to a specific weakness in the surrounding workflow. At the same time, artifact production creates many opportunities for false progress. A revision can memorize visible tasks, overfit a rubric, exploit a judge, or buy gains through additional attempts rather than learn a transferable procedure. Recent studies show that these concerns are empirical, not merely hypothetical (Wang et al., 2026; Huang et al., 2026).

We argue that harness evolution should therefore be evaluated as a controlled search process. Let H_0 be an initial harness, M a fixed foundation model, T a fixed tool interface, B an evolution budget,

and J a trusted evaluator. An evolution algorithm $\mathcal{E}$ receives only policy-approved experience from visible tasks and proposes new harness revisions. A disjoint selection split determines promotion, while the final split remains sealed until a single revision $H_\star$ has been frozen. The principal quantity is the paired generalization gain

$$\Delta_{\text{evo}} = S(H_\star; \mathcal{D}_{\text{final}}) - S(H_0; \mathcal{D}_{\text{final}}), \tag{1}$$

not the best score observed during search. Model identity, task seeds, attempt limits, and resource accounting are matched so that Δ_{evo} can be attributed to the changed harness rather than to a stronger model or more inference.

We operationalize this setting with EvoCowork, an Office-specialized, artifact-centric benchmark. The target design contains 240 tasks, allocating 60 each to presentation, document, spreadsheet, and cross-office workflows and 80/40/120 tasks to feedback, selection, and sealed-final splits. The current implementation freezes the 60-task presentation track, spanning 12 task families and six public sources; **[MOCK: the document, spreadsheet, and cross-office tracks will be completed and frozen before submission.]** Related source files, templates, and near-duplicate workflows share a leakage-group identifier and cannot cross splits. Each completed task is adapted individually from a traceable source into a self-contained execution package with immutable inputs, an explicit output contract, reference artifacts, atomic rubrics, and layered judges.

Task type and task difficulty are separate axes in EvoCowork. Within each modality, fine-grained families describe the operation being performed, such as local editing, reconstruction, synthesis, or cross-file reconciliation. The shared tier names *Easy*, *Medium*, *Hard*, and *Expert* do not impose one structural rubric on incompatible artifacts: each modality defines its own observable progression of scope, evidence dependency, reasoning, artifact structure, and constraint coupling. For presentations, this progression runs from local slide edits to deck-level composition, cross-source synthesis, and end-to-end presentation workflows. A common frozen panel of model–harness configurations then provides an empirical audit through $d_i = 1 - \bar{r}_i$, where $\bar{r}_i$ is the mean task reward under matched budgets and seeds. This shared calibration checks tier ordering and flags boundary cases, but does not replace the modality-specific structural annotation with a label inferred from one agent run.

This design targets an evaluation gap rather than the largest raw task count. Among the representative suites compared in Section 3, Office and workplace benchmarks evaluate fixed agents, editable artifacts, or sandboxed outcomes, whereas harness-evolution benchmarks test learning procedures without making source-grounded, editable Office deliverables their central evaluation object. EvoCowork combines those two sides through replayable artifact tasks, atomic rewards, and a feedback–selection–final boundary designed to distinguish transferable harness improvement from task memorization or additional inference.

We further introduce Group-Relative Harness Search (GRHS), a reference evolution algorithm for the fixed-weight setting. At each round, an updater diagnoses rubric-level failures and generates a group of discrete candidate patches over the allowed harness surface. Candidates are rerun under matched conditions; their improvement, regression, cost, and patch complexity are combined into a scalar utility and standardized within the group. This group-relative advantage is used for discrete selection and for updating a proposal prior—not as a differentiable loss. In contrast to GRPO-based post-training of a dedicated harness engineer (Shao et al., 2024; 2026), GRHS changes no model parameters. Moreover, the updater can be instantiated by the same fixed model and its own mutable harness artifacts. When accepted patches may change those updater artifacts, the mechanism that proposes future revisions also changes, giving a bounded, operational form of harness-level recursive self-improvement. The benchmark controller, judges, sealed tasks, and promotion rule remain immutable.

Our contributions are:

- We introduce an artifact-centric benchmark for experience-driven harness evolution on realistic office tasks, with a 240-task target design across four modalities and a currently frozen 60-task presentation track whose instances retain source-level provenance and replayable deliverables.
- We separate modality-specific task families and structural difficulty from a shared frozen-panel calibration, allowing the four Office modalities to use domain-appropriate difficulty definitions while auditing their empirical ordering under one matched protocol.

- We define a controlled evaluation protocol with leakage-aware feedback–selection–final splits, explicit mutation boundaries, paired fixed-model evaluation, matched test-time-scaling baselines, and accounting for both quality and resource use.
- We present GRHS, a fixed-weight reference method that converts trajectories, artifacts, and rubric feedback into auditable groups of harness revisions, uses group-relative discrete advantage for search, and supports a self-referential updater under an explicit mutation policy. **[MOCK: We will report an evaluation across P base models and Q initial harnesses, including sealed-task gain, transfer, overfitting, cost, and failure modes.]**

2 Related Work

Benchmarks for workplace and computer-use agents have progressively moved from short-form responses toward executable environments and outcome-based evaluation. Styles et al. (2024) evaluate business operations through state changes in a sandboxed workplace, while Xie et al. (2024) study open-ended computer tasks in a real desktop environment. Wang et al. (2024) focus more specifically on long-horizon office automation across multiple applications. These benchmarks expose planning, grounding, and tool-use failures of a submitted agent, but their primary unit of comparison is still the capability of a fixed system at evaluation time.

A complementary line of work emphasizes the quality of files and work products. Ma et al. (2024) introduce real-world spreadsheet-manipulation problems, and Zhu et al. (2026) extend spreadsheet evaluation to end-to-end business workflows. Zhou et al. (2026) evaluate long-horizon office-suite deliverables using fine-grained, code-backed rubrics and economic signals. Tang et al. (2026) study tasks whose solution depends on reasoning across large collections of heterogeneous workspace files. EvoCowork builds on this artifact- and workflow-oriented view, but changes the measured quantity: instead of asking only how capable a frozen agent is, it asks how much a fixed-model harness can improve from a controlled stream of prior task experience.

Agent and language-model programs are increasingly treated as objects that can be optimized. DSPy compiles declarative language-model pipelines against task metrics (Khattab et al., 2023); TextGrad propagates natural-language feedback through compound systems (Yuksekgonul et al., 2024); Automated Design of Agentic Systems searches over code-defined architectures (Hu et al., 2024); and GEPA evolves prompts from execution feedback (Agrawal et al., 2025). Self-referential agents extend the mutable surface to the code that performs future revisions (Yin et al., 2024; Robeyns et al., 2025; Zhang et al., 2025). Recent harness-specific methods make component edits observable and reversible (Lin et al., 2026), constrain and separate component-wise evolution (Zhang et al., 2026a), or recursively refine a prompt-level agent loop from revision history (Lee et al., 2026).

Harness evolution has also become an evaluation target. HarnessOpt-Bench measures an optimizer's held-out gain under an expensive evaluation budget (Ursekar et al., 2026); Evo-Bench evaluates foundation models as harness evolvers across search, Office, and general-agent domains (Huang et al., 2026); and Wang et al. (2026) show that same-task search can overstate improvement and that matched test-time-scaling baselines are necessary. EvoCowork is complementary: it is centered on source-traceable, editable Office deliverables, exposes replayable task packages and artifact-aware judges, and evaluates evolution algorithms under a three-way feedback–selection–final boundary.

GRPO estimates a group-relative advantage for gradient-based policy optimization (Shao et al., 2024). Harness-R1 applies GRPO to post-train a separate model that edits executable harnesses (Shao et al., 2026), while HarnessX uses cross-harness GRPO on the model side of model–harness co-evolution (Chen et al., 2026). Our use of a group-relative statistic is deliberately different: the model remains fixed, the alternatives are discrete harness revisions, and the statistic controls candidate selection and a categorical mutation prior. We therefore describe GRHS as group-relative *search*, not reinforcement learning.

3 THE EVOCOWORK BENCHMARK

3.1 TASK SCOPE AND TAXONOMY

EVOCOWORK treats the work product, rather than a final text response or GUI trajectory, as the unit of evaluation. A task may require creating, editing, analyzing, repairing, or auditing one or more Office artifacts. The 240-task design is balanced across four top-level modalities; Table 1 summarizes the planned coverage and current difficulty distribution. Fine-grained families capture the operation being performed—for example reconstruction, formula authoring, visual redesign, or cross-file reconciliation. The four modalities share the labels *Easy*, *Medium*, *Hard*, and *Expert*, but not a universal difficulty rubric: each modality defines its own observable progression of domain-specific work. For presentations, the progression runs from local slide editing, through deck-level composition and cross-source synthesis, to end-to-end presentation workflows. The operational definitions are given in Appendix A; the corresponding definitions for the other modalities remain intentionally separate. To align these modality-specific labels empirically, every modality is evaluated by the same frozen panel $\mathcal{A}$ of model–harness configurations under matched budgets and a common seed policy. For task i, we summarize panel performance as

$$\bar{r}_i = \frac{1}{|\mathcal{A}|S} \sum_{a \in \mathcal{A}} \sum_{s=1}^{S} r_{i,a,s}, \qquad d_i = 1 - \bar{r}_i, \tag{2}$$

where $r_{i,a,s} \in [0, 1]$ is the task reward and d_i is a descriptive empirical difficulty. This calibration checks aggregate tier ordering and flags boundary cases for audit; it does not replace the structural annotation with a label inferred from one run.

Table 1: Benchmark composition, artifact coverage, and difficulty distribution. Unfinished modality statistics are left blank rather than projected.

	Scale and coverage			Artifact counts		Difficulty distribution			
Modality	Tasks	Families	Sources	Inputs	Outputs	Easy	Medium	Hard	Expert
Presentation	60	12	6	105	67	15	16	17	12
Document	60	–	–	–	–	–	–	–	–
Spreadsheet	60	–	–	–	–	–	–	–	–
Cross-office	60	–	–	–	–	–	–	–	–
Total	**240**	**–**	**–**	**–**	**–**	**–**	**–**	**–**	**–**

For presentation tasks, inputs and outputs are task-package artifact counts; 19 tasks require multiple inputs and five require multiple outputs.

Table 2 separates raw task count from the capabilities that a benchmark can measure. Existing Office and workplace benchmarks provide strong artifact or environment evaluation, while recent learning and harness-optimization benchmarks provide experience or mutation protocols. None combines benchmark-wide editable Office deliverables with isolated task replay, atomic artifact rewards, a three-way evolution boundary, and a self-referential updater track.

Table 2: Representative benchmark capabilities. ✓/△/– indicate full, partial, and no support; P/D/S/X denote the four Office modalities. Counts follow each benchmark's released evaluation unit.

Benchmark	Tasks	Scope	Real source grounding	Editable artifacts	Isolated replay	Atomic reward	Three-way split	Harness mutation	Self-ref. updater
PPT-Eval (Gandhi et al., 2026)	120	P	△	✓	△	✓	–	–	–
OmegaUse-OfficeVal (Zhou et al., 2026)	100	P/D/S	✓	✓	△	✓	–	–	–
OfficeBench (Wang et al., 2024)	300	Multi-app	△	△	✓	△	–	–	–
OSWorld (Xie et al., 2024)	369	Desktop	✓	△	✓	△	–	–	–
SpreadsheetBench (Ma et al., 2024)	912	S	✓	✓	△	△	–	–	–
TheAgentCompany (Xu et al., 2024)	175	Workplace	△	△	✓	✓	–	–	–
STATE-Bench (Microsoft, 2026)	450	Enterprise	–	–	✓	△	△	△	–
HarnessOpt-Bench (Ursekar et al., 2026)	4 targets	General	△	△	✓	△	△	✓	–
Evo-Bench (Huang et al., 2026)	608	S/O/G	△	△	✓	△	△	✓	–
EVOCOWORK (ours)	**240**	**P/D/S/X**	✓	✓	✓	✓	✓	✓	✓

3.2 Source-Grounded Task Construction

Tasks are selected from public, license-compatible sources and adapted one at a time, rather than replaced by newly invented mock problems. The current presentation track draws from PPTArena (Ofengenden et al., 2025), OmegaUse-OfficeVal (Zhou et al., 2026), AutomationBench, TheAgentCompany (Xu et al., 2024), Cowork-Bench, and OfficeBench (Wang et al., 2024). Each adaptation freezes the required inputs, maps the upstream request into an explicit deliverable contract, records checksums and the exact upstream revision, and rebuilds or strengthens the evaluator where necessary. The presentation source inventory and upstream task identifiers are reported in Appendix B. **[MOCK: Source inventories and adaptation records for the document, spreadsheet, and cross-office tracks will be added after those tracks are frozen.]**

3.3 Replayable Task Packages and Artifact-Aware Judges

Each presentation task is packaged as a self-contained, Harbor-compatible unit that exposes only its instruction, immutable input files, execution environment, and output contract to the agent. Its task-specific judge then checks the delivered PPTX for file validity, required editable structures and content, preservation constraints, and, where needed, rendered layout; Golden and targeted negative replays are retained to audit the judge. **[MOCK: The corresponding package and judge specifications for the document, spreadsheet, and cross-office tracks will be added after implementation.]**

3.4 Evolution Splits and Information Boundaries

The feedback split exposes trajectories, artifacts, task scores, and rubric-level diagnostics. The selection split returns only the aggregate signal declared by the protocol, and its queries are metered; it can promote a revision but cannot supply task-level repair instructions. The final split is unavailable until evolution stops. Splitting occurs by leakage group, so shared source documents, templates, entities, or near-duplicate workflows remain in one partition. The benchmark controller also keeps the solver model, updater model, task packages, judges, seeds, and resource accounting outside the writable harness surface. Section 4 gives one algorithm that operates within these boundaries. Appendix C reports the split allocation and information returned at each stage.

4 A Reference Method: Group-Relative Harness Search

4.1 Problem Formulation

Let M_s and M_u denote fixed solver and updater models, T the fixed tool API, and $\mathcal{D} = (\mathcal{D}_{\text{fb}}, \mathcal{D}_{\text{sel}}, \mathcal{D}_{\text{final}})$ the three benchmark splits. At round t, the mutable harness is

$$H_t = \left(H_t^{\text{solve}}, H_t^{\text{update}}, \mathcal{M}_t, q_t\right), \tag{3}$$

where H_t^{solve} controls task execution, H_t^{update} controls how revisions are proposed, $\mathcal{M}_t$ is a versioned evidence memory, and q_t is a categorical prior over mutation operators. The operator set

$$\Omega = \{\text{prompt}, \text{skill}, \text{memory}, \text{middleware}, \text{hook}, \text{tool policy}, \text{workflow code}\} \tag{4}$$

is filtered by the track-specific mutation policy. The trusted control plane is

$$\mathcal{C} = (\mathcal{D}, J, B, \mathcal{P}, \text{Promote}), \tag{5}$$

comprising the tasks, judges, budget, path policy, and promotion rule; it is immutable.

The objective is not to maximize the score seen during search. Given a fixed evolution budget of T_{max} rounds, the algorithm must output one frozen revision $H_\star$ that maximizes expected score on the inaccessible final distribution while controlling regressions and resource use:

$$H_\star = \arg\max_{H \in \mathcal{H}_{\text{visited}}} \mathbb{E}_{q \sim \mathcal{D}_{\text{final}}}[s(q, H)] \quad \text{s.t.} \quad H \models \mathcal{P},\ C(H) \le B. \tag{6}$$

4.2 Failure Evidence and Self-Referential Patch Groups

On a scheduled feedback minibatch $\mathcal{B}_t \subseteq \mathcal{D}_{\text{fb}}$, the current solver produces an experience set

$$X_t = \big\{(q_i, \tau_{t,i}, y_{t,i}, \mathbf{r}_{t,i}, c_{t,i})\big\}_{i \in \mathcal{B}_t}, \tag{7}$$

containing each task, execution trace, artifact, rubric vector, and resource cost. A diagnostic pass clusters recurring failures and binds every proposed change to evidence in X_t. Conditioned on this diagnosis, the current updater samples a sibling group of G patches,

$$\pi_{t,1:G} \sim U_{M_u, H_t^{\text{update}}}\big(\cdot \mid H_t, X_t, \mathcal{M}_t, q_t\big), \qquad H_t^{(g)} = \text{Apply}(H_t, \pi_{t,g}). \tag{8}$$

Every patch carries a mutation manifest, predicted effect, affected components, and rollback point. Syntax, path, dependency, secret-access, and budget checks reject an invalid candidate before it can run.

In the self-referential track, $M_u = M_s$ and H_t^{update} is itself within the allowed mutation surface. An accepted patch may therefore alter the prompt, skills, memory policy, or workflow that generates the next patch group. The update mechanism at round $t+1$ is then a product of earlier updates, which is the operational sense in which we use *harness-level recursive self-improvement*. Freezing H^{update} yields a non-recursive control. In either case, the updater never modifies $\mathcal{C}$; recursion applies to the candidate harness, not to the benchmark or its acceptance criteria.

4.3 Discrete Group-Relative Advantage

All sibling candidates share the same parent, feedback evidence, selection minibatch, seeds, and execution budget. Let $\mathcal{V}_t \subseteq \mathcal{D}_{\text{sel}}$ be the precommitted selection batch. Candidate utility is

$$\begin{aligned} R_{t,g} = S(H_t^{(g)}; \mathcal{V}_t) - S(H_t; \mathcal{V}_t) \\ - \lambda_{\text{reg}}\, \rho_{t,g} - \lambda_{\text{cost}}\, \Delta C_{t,g} - \lambda_{\text{size}}\, \kappa(\pi_{t,g}), \end{aligned} \tag{9}$$

where $\rho_{t,g}$ measures paired task regressions, $\Delta C_{t,g}$ is incremental resource use, and κ penalizes unnecessary patch complexity. Policy-invalid candidates receive $R_{t,g} = -\infty$. For the valid members $\mathcal{G}_t$, GRHS computes

$$A_{t,g} = \frac{R_{t,g} - \mu_t}{\sigma_t + \epsilon}, \qquad \mu_t = \frac{1}{|\mathcal{G}_t|} \sum_{g \in \mathcal{G}_t} R_{t,g}. \tag{10}$$

Here σ_t is the standard deviation within the valid group. If fewer than two candidates remain valid, the relative update is skipped and candidate generation is retried within the original budget. This statistic borrows the within-group baseline of GRPO but is not a policy-gradient loss: no token log-probabilities, clipping ratio, KL term, or parameter update is used. It ranks discrete artifacts and updates the mutation prior. If $o(\pi_{t,g}) \subseteq \Omega$ is the set of operators touched by a patch, then

$$w_{t+1,k} = w_{t,k} \exp\left(\eta \sum_{g \in \mathcal{G}_t} \frac{A_{t,g}\, \mathbf{1}[k \in o(\pi_{t,g})]}{|o(\pi_{t,g})|}\right), \qquad q_{t+1,k} = \frac{w_{t+1,k}}{\sum_j w_{t+1,j}}. \tag{11}$$

Thus successful mutation families become easier to sample without pretending that a source-code patch is a differentiable action.

4.4 Promotion, Rollback, and Final Freezing

The controller selects the candidate with the largest paired-bootstrap lower confidence bound,

$$g^\star = \arg\max_{g \in \mathcal{G}_t} \text{LCB}_\alpha(R_{t,g}). \tag{12}$$

It promotes $H_t^{(g^\star)}$ only if this lower bound exceeds a precommitted margin δ, the regression cap is satisfied, and all policy and budget gates pass; otherwise $H_{t+1} = H_t$. The resulting decision, patch, parent revision, evaluation seeds, score vector, cost, and rollback reason are appended to an immutable evolution ledger and summarized into $\mathcal{M}_{t+1}$. Evolution stops at the budget limit or a predeclared patience rule. The selected revision is then frozen, stripped of feedback and selection access, and evaluated once on $\mathcal{D}_{\text{final}}$ under the same solver conditions as H_0.

5 Experimental Setup

We compare methods that update different parts of the fixed-model agent. Following the shared-protocol comparison in SEAGym (Zheng et al., 2026), we include ACE, TF-GRPO, and AHE as context-, experience-, and harness-level self-evolution baselines (Zhang et al., 2026b; Cai et al., 2025; Lin et al., 2026). We additionally adapt the public Evo-Bench evolver loop (Huang et al., 2026) as an unrestricted policy-harness editing baseline. All methods start from the same seed harness, use the same frozen solver and updater models and benchmark controller, and receive matched feedback, selection-query, rollout, and wall-clock budgets. A frozen seed and budget-matched parallel sampling serve as non-evolution controls (Wang et al., 2026).

6 Results and Analysis

Table 3: Matched-budget final-split results. Frozen H_0 and parallel sampling are non-evolution controls; Evo-Bench denotes its released evolver loop. Scores are mean task rewards and Gain is relative to H_0.

Method	Mutable state	Presentation	Document	Spreadsheet	Cross-office	Overall	Gain
Frozen H_0	None	–	–	–	–	–	–
Parallel sampling (Wang et al., 2026)	Task outputs only	–	–	–	–	–	–
ACE (Zhang et al., 2026b)	Context playbook	–	–	–	–	–	–
TF-GRPO (Cai et al., 2025)	Experience library	–	–	–	–	–	–
AHE (Lin et al., 2026)	Prompt, tools, middleware, memory	–	–	–	–	–	–
Evo-Bench evolver (Huang et al., 2026)	Executable policy harness	–	–	–	–	–	–
GRHS (ours)	**Declared harness surface + updater**	**–**	**–**	**–**	**–**	**–**	**–**

Table 4: Generalization from feedback tasks to selection and sealed final tasks.

Method	Feedback gain	Selection gain	Final gain	Overfitting gap	Final CI (95%)	Retained gain
–	–	–	–	–	–	–
–	–	–	–	–	–	–
–	–	–	–	–	–	–
–	–	–	–	–	–	–

Table 5: Ablation study of the proposed evolution method under matched budgets.

Variant	Final score	Gain	Success rate (%)	Regression rate (%)	Policy violations	Evolution cost ($)
Full GRHS	**–**	**–**	**–**	**–**	**–**	**–**
w/o group-relative advantage	–	–	–	–	–	–
Fixed updater (non-recursive)	–	–	–	–	–	–
w/o held-out promotion	–	–	–	–	–	–
w/o rollback	–	–	–	–	–	–

7 Discussion and Limitations

8 Conclusion

AI Use Statement

Ethics Statement

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts,

Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, Yong Mao, Ke Li, and Xing Sun. Training-free group relative policy optimization. arXiv:2510.08191, 2025. URL https://arxiv.org/abs/2510.08191. Preprint.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. HarnessX: A composable, adaptive, and evolvable agent harness foundry. arXiv:2606.14249, 2026. URL https://arxiv.org/abs/2606.14249. Preprint.

Apurva Gandhi, Vishwas Suryanarayanan, Raja Hasnain Anwar, Firoz Shaik, Shubhang Desai, Thong Q. Nguyen, Muhammad Taqi Raza, Vishal Chowdhary, and Graham Neubig. PPT-Eval: A benchmark for computer-use agents on PowerPoint tasks. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL https://microsoft.github.io/ppteval/.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

Lisheng Huang, Chen Yang, Hao Zhou, Huatong Song, Zongchao Chen, Ran Le, Yang Song, Wayne Xin Zhao, and Tao Zhang. Evo-Bench: Can language models improve agent harness? arXiv:2608.09096, 2026. URL https://arxiv.org/abs/2608.09096. Preprint.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement. arXiv:2607.15524, 2026. URL https://arxiv.org/abs/2607.15524. Preprint.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. arXiv:2604.25850, 2026. URL https://arxiv.org/abs/2604.25850. Preprint.

Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. SpreadsheetBench: Towards challenging real world spreadsheet manipulation. In *Advances in Neural Information Processing Systems*, 2024.

Microsoft. STATE-Bench: Evaluating agents on realistic enterprise tasks. GitHub repository, 2026. URL https://github.com/microsoft/STATE-Bench.

Michael Ofengenden, Yunze Man, Ziqi Pang, Liang-Yan Gui, and Yu-Xiong Wang. PPTArena: A benchmark for agentic PowerPoint editing. *arXiv preprint arXiv:2512.03042*, 2025.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. arXiv:2504.15228, 2025. URL https://arxiv.org/abs/2504.15228. Preprint.

Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories. arXiv:2608.02276, 2026. URL https://arxiv.org/abs/2608.02276. Preprint.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024. URL https://arxiv.org/abs/2402.03300. Preprint.

Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. WorkBench: A benchmark dataset for agents in a realistic workplace setting. *arXiv preprint arXiv:2405.00823*, 2024.

Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Weizheng Wang, Hongzhang Huang, Jun Zhou, Jiachen Song, Shaoli Yu, Jinqi Wang, Zihang Zhou, Hongyi Zhou, Yuting Lv, Jinyang Li, Jiashuo Liu, Ruoyu Chen, Chunwei Liu, GuoLiang Li, Jihua Kang, and Fan Wu. Workspace-Bench 1.0: Benchmarking AI agents on workspace tasks with large-scale file dependencies. *arXiv preprint arXiv:2605.03596*, 2026.

Varun Ursekar, Apaar Shanker, Yash Maurya, Shehab Yasser, Vijay S. Kalmath, Veronica Chatrath, and Yuan Xue. HarnessOpt-Bench: Evaluating LLMs at harness optimization. arXiv:2608.06301, 2026. URL https://arxiv.org/abs/2608.06301. Preprint.

Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. arXiv:2607.12227, 2026. URL https://arxiv.org/abs/2607.12227. Preprint.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. OfficeBench: Benchmarking language agents across multiple applications for office automation. *arXiv preprint arXiv:2407.19056*, 2024.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. arXiv:2412.14161, 2024. URL https://arxiv.org/abs/2412.14161. Preprint.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. arXiv:2410.04444, 2024. URL https://arxiv.org/abs/2410.04444. Preprint.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. arXiv:2505.22954, 2025. URL https://arxiv.org/abs/2505.22954. Preprint.

Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. HarnessCompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses. arXiv:2608.01918, 2026a. URL https://arxiv.org/abs/2608.01918. Preprint.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026b. URL https://openreview.net/forum?id=eC4ygDs02R.

Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. SEAGym: An evaluation environment for self-evolving LLM agents. arXiv:2606.17546, 2026. URL https://arxiv.org/abs/2606.17546. Preprint.

Jingbo Zhou, Yusai Zhao, Qi Bao, Jingjia Cao, Zhenghai Chen, Chang Gao, Kaiqi Guo, Muxin Guo, Mingxuan Li, Xinjiang Lu, Yanru Ma, Yixiong Xiao, Zenghui Zhang, Le Zhang, and Hua Wu. OmegaUse-OfficeVal: Benchmarking LLM agents on long-horizon office-suite tasks with economic grounding. *arXiv preprint arXiv:2607.27155*, 2026.

Jian Zhu, Yuzheng Zhang, Zeyao Ma, Bohan Zhang, Armin Schoepf, Daniel Woloch, Peter Yiliu Wang, Guangyu Robert Yang, Samuel Jacob, Siddharth Nagisetty, Abhiram Chundru, Jean Lin, Spencer Mateega, and Jing Zhang. SpreadsheetBench 2: Evaluating agents on end-to-end business spreadsheet workflows. *arXiv preprint arXiv:2606.29955*, 2026.

A Domain-Specific Difficulty Taxonomies

The four benchmark modalities use the same ordinal tier names, but each modality owns its operational definition. A tier therefore denotes progression within a modality, not an assumption that two artifacts with the same label require identical operations.

A.1 Common Empirical Difficulty Calibration

Structural labels are assigned within each modality before agent evaluation. We then apply one frozen panel of model–harness configurations to every modality, keeping the model and harness versions, task package, judge, seed policy, evaluation budget, and resource accounting fixed for the corresponding calibration run. Equation 2 defines task-level empirical difficulty. For tier ℓ with task set $\mathcal{T}_\ell$, the calibration statistic is

$$\bar{r}_\ell = \frac{1}{|\mathcal{T}_\ell|} \sum_{i \in \mathcal{T}_\ell} \bar{r}_i. \tag{13}$$

We expect $\bar{r}_{\text{Easy}} > \bar{r}_{\text{Medium}} > \bar{r}_{\text{Hard}} > \bar{r}_{\text{Expert}}$ at the tier level; individual tasks may overlap. A systematic inversion triggers a joint audit of the task contract, structural label, judge, and execution trace. A single configuration or run never relabels a task automatically, and an audited label may move only to an adjacent tier when the modality-specific structural rubric supports the change.

The current presentation study is an initial calibration rather than a final cross-model difficulty estimate: all three harnesses use the same Sonnet 5 backbone, and each task contributes one selected valid completion after infrastructure retries are excluded. The final benchmark will use the same calibration procedure for all four modalities and will add multiple model families and repeated seeds before fixing shared empirical tier boundaries.

A.2 Presentation

Presentation difficulty is annotated from five observable properties: operation scope, evidence dependency, reasoning and synthesis, PowerPoint structural depth, and constraint coupling. Slide count alone is not a difficulty criterion. These properties place a task into one of four workflow regimes:

Table A1: Operational difficulty taxonomy for presentation tasks.

Tier	Workflow regime	Operational definition	Representative requirements
Easy	Local editing	A supplied deck receives localized, explicit, and deterministic edits with no external evidence synthesis.	Typography, footer, border, or other isolated slide-object changes.
Medium	Deck-level composition	One deck requires coordinated changes across objects or slides, or manipulation of a bounded native PowerPoint structure.	Reordering slides, editing charts or tables, notes, hyperlinks, or reading order.
Hard	Cross-source synthesis	The task requires substantial reconstruction or deck-wide transformation from visual, tabular, textual, or multiple reference materials while satisfying coupled fidelity constraints.	Image-to-editable diagrams, spreadsheet-to-deck conversion, localization, or template transfer.
Expert	End-to-end orchestration	A business objective requires heterogeneous evidence, long-horizon analysis or reconciliation, and a complete presentation workflow with traceability or multiple deliverables.	Leadership reviews, board briefings, operational analyses, or presentation packages.

The current structural audit assigns 15 tasks to Easy, 16 to Medium, 17 to Hard, and 12 to Expert. Six business-analysis briefings were moved from Hard to Expert because they require end-to-end reconciliation, provenance, and management-facing delivery; three high-effort but bounded deck-production tasks were moved from Expert to Hard. Table A2 reports the resulting pilot calibration.

Table A2: Pilot empirical calibration of the 60 presentation tasks. All rewards and empirical difficulty values are percentages.

Tier	Tasks	Claude Code	OpenCode	OpenHands	Panel mean $\bar{r}_\ell$	$1 - \bar{r}_\ell$
Easy	15	65.73	65.73	72.26	67.91	32.09
Medium	16	19.54	18.88	18.52	18.98	81.02
Hard	17	10.27	8.18	8.99	9.15	90.85
Expert	12	5.78	7.42	7.42	6.87	93.13

The panel uses Claude Code 2.1.251, OpenCode 1.4.14, and OpenHands SDK 1.44.1 with the same Sonnet 5 backbone and frozen task-specific judges. The tier means decrease for all three harnesses. The pooled per-task reward has a descriptive Spearman correlation of $\rho = -0.526$ with ordinal tier. However, 37 of 60 tasks receive zero from all three harnesses, so the pilot has a substantial floor effect and does not establish final cross-model cut points. **[MOCK: Inter-annotator agreement and repeated-seed calibration will be added before the difficulty labels are frozen.]**

A.3 Document

[MOCK: Document-specific difficulty taxonomy.]

A.4 Spreadsheet

[MOCK: Spreadsheet-specific difficulty taxonomy.]

A.5 Cross-Office

[MOCK: Cross-office-specific difficulty taxonomy.]

B Domain-Specific Task Sources and Adaptation Records

B.1 Presentation

The current presentation pool contains 60 individually adapted tasks from six public sources. Table A3 reports the frozen source composition; full commit hashes, input checksums, original requests, licenses, and per-task transformation notes remain inside each released task package.

Table A3: Frozen source inventory for the 60 presentation tasks.

Upstream source	Tasks	License and frozen revision	Upstream task identifiers
PPTArena	31	MIT; `6f26a63` (code), `20fb890` (data)	Cases 001, 003, 004, 014, 015, 016, 021, 023, 025, 027, 028, 029, 030, 031, 036, 037, 041, 044, 047, 050, 053, 059, 061, 064, 067, 068, 071, 074, 082, 083, and 093.
OmegaUse-OfficeVal	17	Apache-2.0; `ffbeecb` (code), `cd6ba6d` (data)	OfficeVal 041, 043, 046, 051, 054, 055, 056, 060, 063, 064, 066, 068, 069, 070, 074, 075, and 076.
AutomationBench	4	MIT; `4a8e106`	Marketing 1115; Finance 4040, 4044, and 4062.
TheAgentCompany	4	MIT; `98b68ef`	`pm-prepare-meeting-with-customers`, `pm-present-gitlab-info-as-ppt`, `hr-internal-tooling-slides`, and `ds-stock-analysis-slides`.
Cowork-Bench	2	Apache-2.0; `d943e75`	`sf-department-budget-ppt` and `ppt-clickhouse-executive`.
OfficeBench	2	Apache-2.0; `b978b80`	Tasks 3-2/subtask-0 and 3-5/subtask-0.
Total	**60**		

Adaptation is performed task by task. Depending on the source, it may freeze remote or application state into immutable files, preserve the original request and assets, replace an online side effect with an explicit local deliverable, reconstruct a reference artifact from source evidence, or strengthen a

weak upstream evaluator with task-specific artifact checks. These changes preserve the source task's business facts and required outcome rather than substituting a newly generated mock scenario.

B.2 Document

[MOCK: Document source inventory and adaptation records.]

B.3 Spreadsheet

[MOCK: Spreadsheet source inventory and adaptation records.]

B.4 Cross-Office

[MOCK: Cross-office source inventory and adaptation records.]

C Evolution Split Allocation and Information Boundaries

All four modalities use the same 20/10/30 feedback–selection–final allocation. Tasks sharing a source document, template, entity, or near-duplicate workflow are assigned to one split through a common leakage group.

Table A4: Benchmark split allocation and information available during evolution.

Split	Total tasks	Per modality	Availability during evolution	Returned information
Feedback (train)	80	20	Visible	Trajectories, artifacts, task scores, and rubric diagnostics
Selection (validation)	40	10	Metered	Protocol-declared aggregate score only
Final (test)	120	30	Sealed	Unavailable until the harness is frozen

D Additional Experimental Tables

Table A5: System configurations, compared evolution methods, and matched budgets.

(a) Base-model, harness, and office-tool configurations

Configuration	Base model	Initial harness	Office tool stack	Context limit	Seeds	Final repeats
Config-A	–	–	–	–	–	–
Config-B	–	–	–	–	–	–
Config-C	–	–	–	–	–	–

(b) Compared evolution methods

Method	Category	Mutation scope	Feedback used	Updater model	Selection rule	Budget tier
Baseline	–	–	–	–	–	–
Prompt-only	–	–	–	–	–	–
Skill+Memory	–	–	–	–	–	–
GRHS (ours)	–	–	–	–	–	–
Oracle	–	–	–	–	–	–

(c) Matched resource budgets

Resource	Feedback phase	Selection phase	Final phase	Per candidate	Per method	Matching rule
Tokens	–	–	–	–	–	–
Wall time	–	–	–	–	–	–
API calls	–	–	–	–	–	–
Compute cost	–	–	–	–	–	–
Human effort	–	–	–	–	–	–

Table A6: Fine-grained evolution gains across task families and difficulty strata.

Modality	Task family	Difficulty	Tasks	Initial score	Final score	Gain
Presentation	–	Easy	–	–	–	–
Presentation	–	Medium	–	–	–	–
Document	–	Easy	–	–	–	–
Spreadsheet	–	Hard	–	–	–	–
Cross-office	–	Medium	–	–	–	–

Table A7: Scaling behavior and cost efficiency across evolution budgets.

Budget tier	Epochs	Candidates	Agent tokens (M)	Updater tokens (M)	Wall time (h)	Cost ($)	Final gain	Gain per $
Small	–	–	–	–	–	–	–	–
Medium	–	–	–	–	–	–	–	–
Large	–	–	–	–	–	–	–	–
X-Large	–	–	–	–	–	–	–	–

Table A8: Transfer and robustness of evolved harnesses under controlled distribution shifts.

Transfer setting	Tasks	Seeds	Source gain	Target gain	Retained gain	CI (95%)
Same modality	–	–	–	–	–	–
Cross modality	–	–	–	–	–	–
New sources	–	–	–	–	–	–
Difficulty shift	–	–	–	–	–	–
Full distribution	–	–	–	–	–	–

Table A9: Failure taxonomy before and after agent-harness evolution.

Failure category	Initial count	Final count	Absolute change	Relative change (%)	Newly introduced	Resolved
Planning errors	–	–	–	–	–	–
Tool misuse	–	–	–	–	–	–
Content errors	–	–	–	–	–	–
Format violations	–	–	–	–	–	–
Citation errors	–	–	–	–	–	–
Quality issues	–	–	–	–	–	–